

Q1. Implement the retrieval stage of a Retrieval-Augmented Generation pipeline using cosine similarity.

You are given one query embedding and m document embeddings, each having dimension d . Compute the cosine similarity between the query and every document.

For document i , use:

$$\text{similarity}[i] = \text{dot}(\text{query}, \text{document}[i]) / (\|\text{query}\| * \|\text{document}[i]\|)$$

Select the k documents with the highest similarities. If two documents have exactly the same similarity, the document with the smaller original zero-based index must rank first.

Apply a numerically stable Softmax to the k selected similarity scores and use those retrieval weights to construct a weighted context vector from the selected document embeddings.

Input Format

m d k

query embedding

m rows of document embeddings

Output Format

Print all m cosine similarities in original document order.

Print the k selected zero-based document indices in ranking order.

Print the k retrieval weights in ranking order.

Print the d -dimensional weighted context vector.

Print floating-point values to exactly four decimal places.

Constraints

- $1 \leq m \leq 50$
- $1 \leq d \leq 20$
- $1 \leq k \leq m$
- The query embedding is non-zero.
- Every document embedding is non-zero.
- Use NumPy only.

Head

```
import numpy as np
```

Body

```
def cosine_similarity(query, document):
    # Compute cosine similarity for one document
    pass

def compute_similarities(query, documents):
    # Return similarities in original document order
    pass

def rank_top_k(similarities, k):
    # Rank by descending similarity, then smaller index on ties
    pass

def retrieval_weights(selected_scores):
```

```

        # Apply numerically stable Softmax
        pass

def weighted_context(documents, indices, weights):
    # Return the weighted sum of selected document embeddings
    pass

```

Tail

```

if __name__ == "__main__":
    m, d, k = map(int, input().split())
    query = np.array(list(map(float, input().split()))), dtype=float)
    documents = np.array([
        list(map(float, input().split()))
        for _ in range(m)
    ], dtype=float)

    similarities = compute_similarities(query, documents)
    indices = rank_top_k(similarities, k)
    selected_scores = similarities[indices]
    weights = retrieval_weights(selected_scores)
    context = weighted_context(documents, indices, weights)

    print(" ".join(f"{value:.4f}" for value in similarities))
    print(" ".join(map(str, indices)))
    print(" ".join(f"{value:.4f}" for value in weights))
    print(" ".join(f"{value:.4f}" for value in context))

```

Q2. Implement the output-processing stage of a multi-class neural network for a batch of samples.

The network provides m rows of logits, where each row corresponds to one sample and contains k class scores. Convert every row independently into Softmax probabilities using a numerically stable implementation. For each sample, first find the maximum logit in that row.

Then compute the stabilised exponential value for each class:

$$e[i] = \exp(z[i] - \max(z))$$

The Softmax probability for each class is:

$$p[i] = e[i] / \sum(e)$$

For the correct target class t , the cross-entropy loss is:

$$\text{loss} = -\ln(p[t])$$

Compute this loss separately for each sample.

The final loss is the mean cross-entropy loss across all m samples.

Your program must output the Softmax probability matrix followed by the mean loss.

Input Format

m k

logits row 1

logits row 2

...

logits row m

target1 target2 ... target m

Where:

- m is the number of samples.
- k is the number of classes.
- Each of the next m lines contains k logits.
- The final line contains the target class index for each sample.
- Class indices are zero-based.

Output Format

Print m rows of Softmax probabilities.

After the probability rows, print the mean cross-entropy loss.

All floating-point values must be printed to exactly four decimal places.

Constraints

- $1 \leq m \leq 20$
- $2 \leq k \leq 20$
- $-10000 \leq \text{logit} \leq 10000$
- $0 \leq \text{target} < k$
- Use the natural logarithm.
- Softmax computation must be numerically stable.

Head

```
import numpy as np
```

Body

```
def stabilize_logits(logits):
    # Stabilise each row independently
    pass

def compute_exponentials(stable_logits):
    # Compute exponential values
    pass

def normalize_rows(exp_values):
    # Convert rows into Softmax probabilities
    pass

def target_probabilities(probabilities, targets):
    # Return probability assigned to each target class
    pass

def mean_cross_entropy(target_probs):
    # Return mean cross-entropy loss
    Pass

Tail

if __name__ == "__main__":
    m, k = map(int, input().split())
```

```
logits = np.array([
    list(map(float, input().split()))
    for _ in range(m)
])

targets = np.array(
    list(map(int, input().split()))
)

stable = stabilize_logits(logits)
exp_values = compute_exponentials(stable)
probabilities = normalize_rows(exp_values)

correct_probs = target_probabilities(
    probabilities,
    targets
)

loss = mean_cross_entropy(correct_probs)

for row in probabilities:
    print(" ".join(
        f"{value:.4f}" for value in row
    ))

print(f"{loss:.4f}")
```

Q3. Implement the forward pass of a LoRA-adapted frozen linear layer using NumPy.

A frozen linear layer has a base weight matrix W of size $d_{\text{out}} \times d_{\text{in}}$. LoRA adds two low-rank matrices A of size $r \times d_{\text{in}}$ and B of size $d_{\text{out}} \times r$.

For input vector x , first compute the frozen base output:

$\text{base} = W @ x$

Then compute the low-rank update:

$\text{update} = (\alpha / r) * (B @ (A @ x))$

The final adapted output is: $\text{output} = \text{base} + \text{update}$

The base matrix W must remain unchanged. No training or gradient calculation is required in this question.

Input Format

d_{in} d_{out} r

x

d_{out} rows of W

r rows of A

d_{out} rows of B

α

Output Format

Print the base output vector.

Print the scaled LoRA update vector.

Print the final adapted output vector.

Print all floating-point values to exactly four decimal places.

Constraints

- $1 \leq d_{\text{in}}, d_{\text{out}} \leq 20$
- $1 \leq r \leq \min(d_{\text{in}}, d_{\text{out}})$
- $\alpha > 0$
- All matrix and vector values are finite decimals.
- W must not be modified.
- Use NumPy only.

Head

```
import numpy as np
```

Body

```
def base_forward(W, x):  
    # Compute the frozen linear-layer output  
    pass  
  
def low_rank_projection(A, x):  
    # Project the input into the low-rank space  
    pass  
  
def lora_update(B, low_rank, alpha, r):  
    # Compute and scale the LoRA update  
    pass
```

```
def lora_forward(W, A, B, x, alpha):
    # Return base output, scaled update, and final output
    pass
```

Tail

```
if __name__ == "__main__":
    d_in, d_out, r = map(int, input().split())
    x = np.array(list(map(float, input().split()))), dtype=float)
    W = np.array([
        list(map(float, input().split()))
        for _ in range(d_out)
    ], dtype=float)
    A = np.array([
        list(map(float, input().split()))
        for _ in range(r)
    ], dtype=float)
    B = np.array([
        list(map(float, input().split()))
        for _ in range(d_out)
    ], dtype=float)
    alpha = float(input())

    base, update, output = lora_forward(W, A, B, x, alpha)

    print(" ".join(f"{value:.4f}" for value in base))
    print(" ".join(f"{value:.4f}" for value in update))
    print(" ".join(f"{value:.4f}" for value in output))
```

Q4. Implement a convolutional feature-extraction operation for a grayscale image using **zero padding**.

You are given an $H \times W$ image, a $KH \times KW$ kernel, padding size P , and a bias. Surround the image with P rows and columns of zeros before applying the kernel.

Use:

- stride = 1
- zero padding = P
- CNN cross-correlation convention; do not flip the kernel
- ReLU activation after adding the bias

The output dimensions are:

$$OH = H + 2P - KH + 1$$

$$OW = W + 2P - KW + 1$$

For every valid location, compute the sum of element-wise products between the padded image patch and kernel, add the bias, and apply:

$$\text{ReLU}(x) = \max(0, x)$$

Return the final feature map.

Input Format

$H \ W$

image row 1

...

image row H

$KH \ KW$

kernel row 1

...

kernel row KH

P

bias

Output Format

First print:

OH OW

Then print the resulting feature map.

Constraints

- $1 \leq H, W \leq 20$
- $1 \leq KH, KW \leq 10$
- $0 \leq P \leq 5$
- Output dimensions will always be positive.
- Input, kernel, padding, and bias values are integers.

Head

```
import numpy as np
```

Body

```
def pad_image(image, padding):  
    # Apply zero padding  
    pass  
  
def output_dimensions(H, W, KH, KW, padding):  
    # Calculate output dimensions  
    pass  
  
def kernel_response(patch, kernel):  
    # Calculate one convolution response  
    pass  
  
def raw_feature_map(image, kernel, padding):  
    # Generate raw padded convolution feature map  
    pass  
  
def apply_bias_relu(feature_map, bias):  
    # Add bias and apply ReLU  
    Pass
```

Tail

```
if __name__ == "__main__":
```

```
H, W = map(int, input().split())
```

```
image = np.array([  
    list(map(int, input().split()))  
    for _ in range(H)  
])
```

```
KH, KW = map(int, input().split())
```

```
kernel = np.array([  
    list(map(int, input().split()))  
    for _ in range(KH)  
])
```

```
padding = int(input())  
bias = int(input())
```

```
raw = raw_feature_map(  
    image,  
    kernel,  
    padding  
)
```

```
output = apply_bias_relu(  
    raw,  
    bias  
)
```

```
OH, OW = output.shape  
print(OH, OW)
```

```
for row in output:  
    print(" ".join(  
        map(str, row.astype(int))  
    ))
```

Q5. Implement a perceptron prediction pipeline for multiple input samples. For each sample, first multiply every input feature by its corresponding weight. Sum these weighted values, add the bias, and apply the perceptron step activation to obtain the predicted class.

For an input sample $X = [x_1, x_2, \dots, x_d]$, weights $W = [w_1, w_2, \dots, w_d]$, and bias b , compute:

$$z = (x_1 * w_1) + (x_2 * w_2) + \dots + (x_d * w_d) + b$$

The perceptron prediction is:

- 1 if $z \geq 0$
- 0 if $z < 0$

Complete the five functions provided in the boilerplate so that the entire prediction pipeline works correctly.

Input Format

n d

$x_{11} x_{12} \dots x_{1d}$

$x_{21} x_{22} \dots x_{2d}$

...

$x_{n1} x_{n2} \dots x_{nd}$

$w_1 w_2 \dots w_d$

bias

Where:

- n is the number of input samples.
- d is the number of features in each sample.
- The next n lines contain the input samples (each with d values).
- The next line contains the d weights.
- The last line contains the bias value.

Output Format

Print the predicted classes for all n samples, separated by spaces, on a single line:

$p_1 p_2 \dots p_n$

Constraints

- $1 \leq n \leq 100$
- $1 \leq d \leq 20$

- Input values, weights, and bias may be integers or decimal (floating-point) values.
- A linear score exactly equal to 0 must be classified as 1.

Head

```
import numpy as np
```

Body

```
def weighted_products(sample, weights):
    # Step 1: Return feature-wise weighted products
    pass

def linear_score(products, bias):
    # Step 2: Sum the weighted products and add bias
    pass

def step_activation(score):
    # Step 3: Return 1 if score >= 0, otherwise 0
    pass

def predict_one(sample, weights, bias):
    # Step 4: Predict one sample
    pass

def predict_batch(X, weights, bias):
    # Step 5: Predict all samples
    Pass
```

Tail

```
if __name__ == "__main__":
    n, d = map(int, input().split())

    X = np.array([
        list(map(float, input().split()))
        for _ in range(n)
    ])

    weights = np.array(
        list(map(float, input().split()))
    )

    bias = float(input())
```

```
predictions = predict_batch(X, weights, bias)
```

```
print(" ".join(map(str, predictions)))
```

Q6. Implement a tool-routing decision stage for an Agentic AI system using NumPy.

An agent receives one task-requirement vector of dimension d and has access to m tools. Each tool is represented by a non-zero capability vector of the same dimension.

Compute the cosine similarity between the task vector and every tool capability vector. The supplied availability mask determines which tools may be selected: 1 means available and 0 means unavailable.

Choose the available tool with the highest cosine similarity. If multiple available tools have exactly the same score, choose the tool with the smaller zero-based index.

If no tool is available, or if the best available score is strictly less than the supplied threshold, return -1 to indicate that the agent should not call a tool.

Input Format

m d

task requirement vector

m rows of tool capability vectors

availability mask containing m values

threshold

Output Format

First print the cosine-similarity score for every tool in original tool order.

Then print the selected zero-based tool index, or -1 if no tool should be called.

Print similarity values to exactly four decimal places.

Constraints

- $1 \leq m \leq 20$
- $1 \leq d \leq 20$
- The task vector is non-zero.
- Every tool capability vector is non-zero.
- Each availability value is either 0 or 1.
- $-1 \leq \text{threshold} \leq 1$
- Use NumPy only.

Head

```
import numpy as np
```

Body

```
def cosine_score(task, capability):
    # Compute cosine similarity for one tool
    pass

def score_tools(task, capabilities):
    # Return scores for all tools in original order
    pass

def available_candidates(scores, availability):
    # Return the indices of tools that may be selected
    pass
```

```

def choose_tool(scores, candidates, threshold):
    # Select the best tool with the required tie-breaking rule
    pass

def route_task(task, capabilities, availability, threshold):
    # Return all scores and the selected tool index
    pass

```

Tail

```

if __name__ == "__main__":
    m, d = map(int, input().split())
    task = np.array(list(map(float, input().split()))), dtype=float)
    capabilities = np.array([
        list(map(float, input().split()))
        for _ in range(m)
    ], dtype=float)
    availability = np.array(list(map(int, input().split()))), dtype=int)
    threshold = float(input())

    scores, selected = route_task(
        task, capabilities, availability, threshold
    )

    print(" ".join(f"{value:.4f}" for value in scores))
    print(selected)

```

Q7. Implement sinusoidal positional encoding for a sequence of token embeddings using NumPy.

A sequence contains n token embeddings, each with dimension d . Generate a positional-encoding matrix PE of size $n \times d$ and add it element-wise to the supplied embedding matrix X .

For position pos and integer $i \geq 0$, use:

$$PE[pos, 2i] = \sin(pos / 10000^{(2i/d)})$$

If dimension $2i + 1$ exists, use:

$$PE[pos, 2i + 1] = \cos(pos / 10000^{(2i/d)})$$

The final encoded sequence is: $encoded = X + PE$

Input Format

n d

n rows of X

Output Format

Print the $n \times d$ positional-encoding matrix first.

Then print the $n \times d$ encoded sequence matrix.

Print all floating-point values to exactly four decimal places.

Constraints

- $1 \leq n \leq 20$
- $1 \leq d \leq 20$
- Input embeddings may contain positive, negative, or zero values.
- Positions are zero-based.
- Use the stated sinusoidal formulas exactly.
- Use NumPy only.

Head

```
import numpy as np
```

Body

```
def position_angle(position, pair_index, d):
    # Return position / 10000^(2*pair_index/d)
    pass

def build_position_row(position, d):
    # Build one positional-encoding row
    pass

def positional_encoding(n, d):
    # Build the complete n x d positional-encoding matrix
    pass

def encode_sequence(X, PE):
    # Add positional encoding to the token embeddings
    pass
```

Tail

```
if __name__ == "__main__":
```

```
n, d = map(int, input().split())
X = np.array([
    list(map(float, input().split()))
    for _ in range(n)
], dtype=float)

PE = positional_encoding(n, d)
encoded = encode_sequence(X, PE)

for row in PE:
    print(" ".join(f"{value:.4f}" for value in row))
for row in encoded:
    print(" ".join(f"{value:.4f}" for value in row))
```

Q8. Implement convolution over a grayscale image using a configurable stride.

Given an $H \times W$ image, a $KH \times KW$ kernel, stride S , and bias, calculate the convolution feature map using:

- no padding;
- CNN cross-correlation convention, so the kernel is not flipped;
- the specified stride;
- bias addition followed by ReLU activation.

The output dimensions are:

$$OH = \lfloor (H - KH) / S \rfloor + 1$$

$$OW = \lfloor (W - KW) / S \rfloor + 1$$

For every valid kernel position, calculate the element-wise product sum. Add the bias and apply:

$$\text{ReLU}(x) = \max(0, x)$$

Input Format

H W

image rows

KH KW

kernel rows

stride

bias

Output Format

First print:

OH OW

Then print the resulting feature map.

Constraints

- $1 \leq H, W \leq 20$
- $1 \leq KH \leq H$
- $1 \leq KW \leq W$
- $1 \leq \text{stride} \leq 5$
- Image, kernel, and bias values are integers.

Head

```
import numpy as np
```

Body

```
def output_dimensions(  
    H, W, KH, KW, stride  
):  
    # Calculate output dimensions  
    pass
```

```
def extract_patch(  
    image, row, col,  
    KH, KW, stride  
):  
    # Extract the required patch  
    pass
```

```
def kernel_response(patch, kernel):  
    # Calculate one kernel response  
    pass
```

```
def strided_feature_map(  
    image, kernel, stride  
):  
    # Generate the raw feature map  
    pass
```

```
def apply_bias_relu(feature_map, bias):  
    # Add bias and apply ReLU  
    pass
```

Tail

```
if __name__ == "__main__":  
    H, W = map(int, input().split())  
  
    image = np.array([  
        list(map(int, input().split()))  
        for _ in range(H)  
    ])  
  
    KH, KW = map(int, input().split())
```

```
kernel = np.array([
    list(map(int, input().split()))
    for _ in range(KH)
])
```

```
stride = int(input())
bias = int(input())
```

```
raw = strided_feature_map(
    image, kernel, stride
)
```

```
output = apply_bias_relu(
    raw, bias
)
```

```
OH, OW = output.shape
print(OH, OW)
```

```
for row in output:
    print(" ".join(
        map(str, row.astype(int))
    ))
```

Q9. Implement scaled dot-product self-attention from scratch using NumPy.

You are given query, key, and value matrices for a sequence containing n tokens, with each token represented using d features.

Compute the scaled attention-score matrix as:

$\text{scores} = (Q @ K.T) / \sqrt{d}$

Apply a numerically stable row-wise Softmax to the score matrix. For each row, subtract the maximum value before exponentiation.

Use the resulting attention weights to compute the contextual representation:

$\text{context} = \text{attention_weights} @ V$

Do not use any built-in attention or neural-network implementation.

Input Format

n d

n rows of Q

n rows of K

n rows of V

Output Format

Print the $n \times n$ attention-weight matrix first.

Then print the $n \times d$ contextual-output matrix.

Print all floating-point values to exactly four decimal places.

Constraints

- $1 \leq n \leq 10$
- $1 \leq d \leq 10$
- Q , K , and V contain finite decimal values.
- Scaling must use $\sqrt{d}$.
- Softmax must be applied independently to every row.
- Use NumPy only.

Head

```
import numpy as np
```

Body

```
def scaled_scores(Q, K):
    # Compute Q @ K.T and apply the scaling factor
    pass

def stabilize_rows(scores):
    # Subtract the maximum value from each row
    pass

def softmax_rows(stable_scores):
    # Convert each row into attention probabilities
    pass

def attention_output(attention_weights, V):
```

```
# Compute the contextual representation
pass
```

```
def scaled_dot_product_attention(Q, K, V):
    # Return attention weights and contextual output
    pass
```

Tail

```
if __name__ == "__main__":
    n, d = map(int, input().split())
    Q = np.array([
        list(map(float, input().split()))
        for _ in range(n)
    ], dtype=float)
    K = np.array([
        list(map(float, input().split()))
        for _ in range(n)
    ], dtype=float)
    V = np.array([
        list(map(float, input().split()))
        for _ in range(n)
    ], dtype=float)

    weights, output = scaled_dot_product_attention(Q, K, V)

    for row in weights:
        print(" ".join(f"{value:.4f}" for value in row))
    for row in output:
        print(" ".join(f"{value:.4f}" for value in row))
```

Q10. Implement a bipolar perceptron classifier for multiple input samples.

For each input sample:

$X = [x_1, x_2, \dots, x_d]$
with weights:

$W = [w_1, w_2, \dots, w_d]$

and bias b , calculate the score as:

$\text{score} = (x_1 * w_1) + (x_2 * w_2) + \dots + (x_d * w_d) + b$

Unlike a binary perceptron that produces 0 and 1, this classifier uses bipolar outputs:

- Return 1 if $\text{score} \geq 0$
- Return -1 if $\text{score} < 0$

Complete the functions provided in the boilerplate so that all input samples are classified correctly.

Input Format

$n \ d$

$x_{11} \ x_{12} \ \dots \ x_{1d}$

$x_{21} \ x_{22} \ \dots \ x_{2d}$

...

$x_{n1} \ x_{n2} \ \dots \ x_{nd}$

$w_1 \ w_2 \ \dots \ w_d$

bias

Where:

- n is the number of input samples.
- d is the number of features in each sample.
- The next n lines contain the input samples.
- The following line contains the d weights.
- The final line contains the bias.

Output Format

Print the predicted classes separated by spaces.

$p_1 \ p_2 \ \dots \ p_n$

Constraints

- $1 \leq n \leq 100$
- $1 \leq d \leq 20$

- Input values, weights, and bias may be integers or decimals.
- A score exactly equal to 0 must produce class 1.

Head

```
import numpy as np
```

Body

```
def weighted_products(sample, weights):
    # Return feature-wise weighted products
    pass
```

```
def linear_score(products, bias):
    # Return weighted sum after adding bias
    pass
```

```
def bipolar_activation(score):
    # Return either 1 or -1
    pass
```

```
def predict_one(sample, weights, bias):
    # Predict one sample
    pass
```

```
def predict_batch(X, weights, bias):
    # Predict all samples
    Pass
```

Tail

```
if __name__ == "__main__":
    n, d = map(int, input().split())

    X = np.array([
        list(map(float, input().split()))
        for _ in range(n)
    ])

    weights = np.array(
        list(map(float, input().split()))
    )

    bias = float(input())
```

```
predictions = predict_batch(  
    X, weights, bias  
)  
  
print(" ".join(map(str, predictions)))
```